Ders 7 Genişletilmiş Notu: Dinamik ve Canonical Huffman, MNP5/MNP7 ve Faks Sıkıştırma

Ders 7 Genişletilmiş Notu: Dinamik ve Canonical Huffman, MNP5/MNP7 ve Faks Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Huffman Ağacında Sık Yapılan Hatalar
- 2. Statik, Yarı Dinamik ve Dinamik Huffman
- 3. Dinamik Huffman ve Escape Kodu
- 4. Dinamik Ağacın Değişimi ve Varyans
- 5. Dinamik Huffman'da Aksayan Taraflar: Sayıcı ve Kod Taşması
- 6. Huffman Algoritmasının Basit Bir Versiyonu
- 7. Canonical Huffman Kodlama
- 8. Modemlerde Sıkıştırma: MNP5
- 9. MNP7: İki Boyutlu Dinamik Huffman
- 10. Bilinen Algoritmaların Türevleri
- 11. Faks Sıkıştırma: Standartlar ve Çözünürlük
- 12. Grup 3 ve Modified Huffman
- 13. Grup 3'te Hata Bulma ve Tek Boyutlu Kodlama
- Örnek Sorular

1. Huffman Ağacında Sık Yapılan Hatalar

Hoca derse, önceki hafta anlatılan Huffman kodlamasıyla ilgili gelen sorulardan gördüğü iki hatayı hatırlatarak başladı.

Birinci hata genişletilmiş dizinin oluşturulmasındadır. Dizi genişletilirken bulunan her toplam **sıralamayı bozmayacak şekilde** yerleştirilir. Gelen bir örnekte 6 ile 7 toplanıp 13 bulunmuş, ama 13 sayısı 14'ten sonraya konmuştu. Doğrusu 13'ün 8 ile 14 arasına yazılmasıdır. Bu küçük bir dikkatsizlik gibi görünse de ağaç farklı oluşur, ağaç farklı oluşunca kodlar farklı çıkar ve arada prefix kod bile oluşmayabilir. Eşit değerlerde de her zaman aynı taraf (örneğin hep sağ taraf) tercih edilmelidir.

İkinci hata ağacın kurulmasındadır. En sonda okunan değer (örnekte 60) köke yerleştirilir; 60, 25 ile 35'in toplamıdır. Yerleştirirken dizi sondan okunur ve önceki derste ilk okunan sola, ikinci okunan sağa konmuştu. Hoca "Bu şart mı?" diye sordu ve cevabı verdi: şart değildir. İlk okunan sağa, ikincisi sola da konabilir. Ama hangi kural seçildiyse her seviyede aynı uygulanmalıdır. Bir seviyede ilk okunanı sola, başka bir seviyede sağa koyarsanız çıkan kodlar hatalı olur.

> **Hocanın vurguları:** "Aynı şeyi uygulamak zorundasın ki prefix kodlar oluşabilsin. Bunlar çok önemli."

2. Statik, Yarı Dinamik ve Dinamik Huffman

Hoca, önceki derste başlanan üç Huffman türünü slayt üzerinden yeniden ele aldı (ekrandan, 00:10:00).

2.1 Statik Huffman

Slayttaki tanım: bir derlem içerisinde yer alan tüm sembollerin kullanım sıklıkları çıkarıldıktan sonra her sembol için Huffman kod değerleri oluşturulur. Gelen her doküman için aynı kod değerleri kullanılır; doküman içerisindeki sembollerin kullanım sıklıkları dikkate alınmaz.

Hoca bunu şöyle anlattı: biri bizim yerimize bir korpus alır, o dile ait bütün karakterlerin olasılıklarını çıkarır, Huffman ağacını ve kodları oluşturur ve "senin kullanabileceğin Huffman kodları bunlar" diye elimize verir. Biz de en çok kullanılan sembole bit uzunluğu en küçük olan kodu atarız. Kodlar elimizdeki içeriğe göre hazırlanmadığı için performans diğer türlere göre elbette düşük olur.

2.2 Yarı dinamik Huffman

Slayttaki tanım: kodlanacak her doküman üzerinde iki aşamadan oluşan bir işlem yapılır. İlk olarak doküman baştan sona taranarak sembollerin kullanım sıklıkları çıkarılır. İkinci adımda Huffman kod değerleri elde edilir.

Önceki derste anlatılan yöntem aslında yarı dinamiğe uygundur: her doküman için o dokümandaki sembollerin frekanslarına bakılarak farklı kodlar çıkarılır. Önce doküman bir kez taranıp frekanslar çıkarılır, ağaç kurulur, sonra doküman yeniden okunur ve okunurken kodlanır. Hoca karşılaştırma için şöyle bir örnek verdi: statik Huffman ile %55 bir başarı elde ediliyorsa yarı dinamik ile %70'lik bir başarı elde edilebilir.

2.3 Dinamik Huffman

Slayttaki tanım: hızın önemli olduğu uygulamalarda kullanılır. Hızın önemli olması, verinin bir noktadan başka bir noktaya (hocanın örneğiyle buradan Ankara'da bir noktaya) sıkıştırılarak iletilmesi demektir. Kimse göndericinin önceden oturup ağacı oluşturmasını beklemez; gönderim sırasında iki taraf ağacı karşılıklı olarak aynı anda oluşturur. Dinamik Huffman'ın başarısı da yüksektir ama hoca bunun da değişkenlik gösterdiğini belirtti; asıl tercih sebebi iletim anında kullanılabilmesidir. Algoritma çok daha karmaşıktır.

3. Dinamik Huffman ve Escape Kodu

3.1 Slayttaki kurallar

Slayttaki maddeler şunlardır (ekrandan, 00:10:00):

- Encoder ve decoder senkronize çalışarak aynı anda kodlama ağacını oluşturur.
- Başlangıçta ağaç boştur. Okunan her yeni sembol ile ağaç yeniden oluşturulur.
- Input stream'den ilk sembol okunur ve kodlanmadan output stream'e yazılır.
- Sembol ağaca eklenir ve kendisine bir kod atanır.
- Daha sonraki adımlarda bu sembol ile karşılaşılırsa önce sembolün kodu gönderilir, sonra da frekans değeri bir artırılır.
- Decoder'ın bilmesi gereken, gönderilen sembolün kodlanarak mı yoksa kodlanmadan mı gönderildiğidir. Kodlanmamış sembolden önce escape kodu gönderilir.
- Decoder, escape kodundan sonra gelen kodun ASCII koda sahip bir sembol olduğunu anlar.
- Escape kodun frekans değeri her zaman 1'dir ve değişmez. Ancak her yeni sembol ağaca eklendikçe kodu değişir.

İlk gelen sembolün henüz bir kodu olmadığı için ASCII olarak gönderilir, sonra ağaca eklenir ve kod oluşur. O sembol bir daha geldiğinde artık kod değeriyle gönderilir ve ağaçtaki frekansı değiştirilir; ağacın değişmesi gerekiyorsa bu değişiklik de yapılır. Decoder'ın, gelen bitlerin bir sembolün kodu mu yoksa ASCII kodu mu olduğunu anlaması gerekir. Bu yüzden ağaç bir escape koduyla başlar.

Escape'in frekansı neden değişmez? İngilizce için 26 karakter düşünülürse, her karakter yalnız ilk kez geldiğinde escape gönderilir; bir sembole kod atandıktan sonra o sembol için escape bir daha kullanılmaz. Escape'e eninde sonunda ihtiyaç kalmaz.

3.2 Örnek: `a a b c`

Hoca slayttaki "Escape kodu ve ağaç üzerindeki değişimi" şekli üzerinde `a a b c` girişini adım adım çizdi (ekrandan, 00:20:00). Bir tarafta encoder, öbür tarafta decoder bekliyor ve ikisinde de yalnız escape düğümü var.

1. **İlk a:** a ağaçta yok. Encoder önce escape kodunu, arkasından a'nın ASCII değerini gönderir. Decoder escape'i görünce "ağaçta olmayan bir sembol geldi, arkadaki ASCII kodudur" der, a'yı output stream'e yazar. İki taraf da a'yı ağacın sol koluna ekler. a'nın frekansı 1, sol tarafta olduğu için kodu `0`'dır.
2. **İkinci a:** a ağaçta var ve kodu `0`. Encoder `0` gönderir. Decoder bunun escape olmadığını görür; ağaçta kodu `0` olan sembol a'dır, a'yı yazar. İki taraf da a'nın frekansını 2 yapar. Ağaçta yalnız iki sembol olduğu için ikisi de bir bitle kodlanabilir; değişime gerek yoktur.
3. **b:** b ağaçta yok. Escape ve b'nin ASCII kodu gönderilir; decoder b'yi yazar. Frekanslar artık $a = 2$, escape $= 1$, $b = 1$'dir ve ağaç yeniden düzenlenir. En sık gelen a bir bitle kalır: `a = 0`, `b = 10`, escape `= 11`. Escape'in kodu değişmiştir. Aynı ağaç decoder tarafında da kurulur.
4. **c:** c ağaçta yok. Escape + c'nin ASCII kodu gönderilir; iki taraf da c'yi (frekans 1) ağaca ekler ve escape bir seviye daha aşağı iner.

İşlem input bitene kadar devam eder. Ağaç her büyüdüğünde escape aşağı seviyelere iner ve kodu sürekli değişir. Herhangi bir sembolün kodu da ağaç değiştikçe değişebilir: b'nin kodu bir an `10` iken sonra başka bir değer olabilir. Bu yüzden bir doküman başındayken a bir kodla, ortasında başka bir kodla, sonunda yine başka bir kodla kodlanabilir.

Bunun hiçbir önemi yoktur, çünkü dinamik Huffman'da header bilgisi yoktur. İki taraf ağacı aynı anda oluşturduğu için o anda a'nın hangi koda sahip olduğunu ağaç üzerinden zaten görür. "a'nın farklı kod değerlerini nasıl göndereceğim" diye bir problem de olmaz.

4. Dinamik Ağacın Değişimi ve Varyans

4.1 Eşit toplamın yeri ve varyans

Önceki derste, toplanan iki değer dizide var olan bir değere eşitse (örneğin 7) onun soluna ya da sağına yazılabileceği söylenmişti. Dinamik Huffman kullanılmıyorsa ister sola ister sağa yazılsın, hiçbir şey fark etmez, çünkü entropi aynı çıkar. Ama dinamik Huffman kullanılıyorsa **varyansı küçük olan** yerleşim seçilmelidir.

Varyansın büyük olması, sembollerin bit uzunlukları arasında bariz farklılıklar olması demektir: 1 bit, ardından 6 bit, 7 bit, 10 bit, 17 bit gibi. Varyansın küçük olması ise uzunlukların birbirine yakın olmasıdır: 2 bit, 3 bit, 3 bit, 5 bit gibi.

Hocanın verdiği örnek: a 17 bit, b ve c 3 bit olsun. Gönderici önce 17 bitlik a'yı, arkasından 3 bitlik c'yi gönderir. Alıcı 17 bitlik kodu çözene kadar 3 bitlik kod gelir ve "beni de çöz" diye sıraya girer; arkasından bir 3 bitlik daha gelir. Kodların bit uzunlukları arasındaki mesafe büyük olduğu için habire çözülmeyi bekleyen kodlar oluşur. Varyans küçük olsaydı, örneğin 4 bitlik bir kod gelseydi, alıcı 3 bitlik kod gelene kadar onu çözer ve bunlar "ucu ucuna" yetişirdi; belki küçük bir buffer alanı yeterdi. Varyansı büyük bir kod seçildiğinde buffer alanının çok büyük seçilmesi gerekir. Hızın önemli olduğu durumlarda bunu kimse tercih etmez; bu yüzden varyansı küçük kodlar tercih edilmelidir.

4.2 Ağacın adım adım değişmesi

Hoca, belirli bir aşamaya kadar gelmiş bir ağaç üzerinden dinamik Huffman'da ağacın nasıl değiştiğini gösterdi (ekrandan, 00:26:30 ve 00:29:40).

Başlangıç durumu (kök 16). Bugüne kadar A bir kez, B, C ve D ikişer kez, E dokuz kez gelmiştir. En sık gelen E olduğu için bir bitle kodlanmıştır. Ara değerler genişletilmiş dizideki gibi bulunur: $A + B = 1 + 2 = 3$, $C + D = 2 + 2 = 4$, $3 + 4 = 7$, $7 + 9 = 16$. Sol dallar 0, sağ

dallar 1 alınınca `A = 000`, `B = 001`, `C = 010`, `D = 011`, `E = 1` olur. A ile B, C ve D üçer bitlik, E bir bitliktir. Hoca bilgi teorisini hatırlattı: en sık gelen olayın taşıdığı bilgi daha azdır ve daha az bitle gönderilir.

İkinci A (kök 17). A'nın frekansı 2 olur. $2+2=4$, öbür taraf zaten 4, $4+4=8$; 9 hâlâ 8'den büyüktür; kök $8+9=17$. Bir A gelmesi ağacı değiştirmedi.

Üçüncü A (kök 18). A'nın frekansı 3 olur. Büyük frekanslı değerler sağ tarafa yerleştiği için ağacın şekli değişir. Sıralama 2, 2, 2, 3 olduğundan ağaç yeniden kurulur: D ile B toplanıp 4, C ile A toplanıp $2+3=5$, $4+5=9$; 9 ile E'nin 9'u arka arkaya gelir, değişikliğe gerek yoktur; kök 18. Hoca, değerleri sıralayıp (2, 2, 2, 3, 9) ağacı baştan oluştursaydık da bu ağacın çıkacağını belirtti.

Dördüncü A (kök 19). A'nın frekansı 4 olur. $2+4=6$, öbür taraf 4, $4+6=10$; E ise 9'dur. Büyük olanlar sağ tarafa geldiği için bu seviyede 10 ile 9'un yerleri değişir: artık E sol tarafta, 10'luk alt ağaç sağ taraftadır. A, B, C, D ve E'nin kod değerleri birdenbire değişir.

Beşinci A (kök 20). Ağaç: E (9) solda; sağda 11'lik düğüm, onun altında A (5) ve 6'lık düğüm; 6'nın altında C (2) ve 4'lük düğüm; 4'ün altında D (2) ve B (2). Toplamlar $2+2=4$, $4+2=6$, $6+5=11$, $9+11=20$'dir. Bu ağaçta kodlar yukarıdan okunur; B'nin kodu dört tane 1'dir: 1111.

> **Ders sonrası not:** Slayttaki dördüncü A'ya ait son ağaçta A'nın frekansı (5) yazılı görünür; oysa ara düğüm 10 ve kök 19, A = 4 ile tutarlıdır. A = 5 durumu bir sonraki şekildeki (kök 20) ağaçtır.

5. Dinamik Huffman'da Aksayan Taraflar: Sayıcı ve Kod Taşması

Slaytta "Bu algoritmada aksayan taraflar nelerdir?" sorusu ve iki cevabı vardır (ekrandan, 00:30:00).

5.1 Sayıcının taşması

Slayttaki metin: frekans değerinin tutulduğu değişken işaretsiz 16 bit olsun. Bu değişkende $2^{16}-1=65535$ değer tutabiliriz. Taşmanın oluşmaması için kökteki frekans değeri kontrol edilir. Maksimum sayı değerine ulaşıldığında yeniden ölçeklendirme yapılır: yaprakların değerlerinin yarısı alınır (integer bölme), ara düğümler ve kök tekrardan hesaplanır.

Hoca bunu değişken tipleri üzerinden açıkladı. Toplamların yazıldığı değişken byte ise en fazla 255 frekans yazılabilir; word ise 16 bitlik bir sayı. Doküman çok büyükse frekans değerleri byte'ı ya da word'ü taşar. Örneğin bir değer 255'e gelmişken bir A daha gelirse 256 byte'a sığmaz; değer 255 olarak kalır ve yeni gelen frekanslar kaybolur. Bu yüzden doküman büyükse algoritmada sayaç kontrol edilmelidir: en tepedeki köke bakılır, kök kendisi için ayrılan alanın taşıyabileceği en büyük sayıya gelmişse bütün düğüm değerleri yarıya bölünür. Ağaç değişmez, yine aynı ağaçtır.

Slayttaki örnek: yapraklar 155, 155, 310, 310; ara düğümler 310 ve 620; kök 930. Yapraklar yarıya bölününce 77, 77, 155, 155 olur; ara düğümler yeniden toplanarak $77+77=154$ ve $155+155=310$, kök $154+310=464$ bulunur. Kökün doğrudan yarısı $930/2=465$ olurdu; integer bölmeden dolayı yeniden hesaplanan kök 464'tür (python ile kontrol edildi).

> **Ders sonrası not:** Hoca 00:41:03'te (basit versiyonu anlatırken) word için sınırı "32 bin bilmem ne" diye söyledi. Slayttaki işaretsiz 16 bitlik değişkenin sınırı 65535'tir; 32767, işaretli 16 bitlik bir tamsayının sınırıdır.

5.2 Kod taşması

Slayttaki metin: kodların tutulduğu alan 16 bit ve ağacın seviyesi de 16'dan fazla ise ya bu alan genişletilir ya da linkli liste kullanılır.

Hoca örnek verdi: B'nin kodu 16 bit uzunluğunda olsun. Kodlar bir byte ya da word değişkeninde tutuluyorsa, byte'ta 8 biti, word'de 16 biti geçen kodların geri kalanı nerede tutulacak? Ya değişken genişletilir ya da linkli liste kullanılır. Linkli listede örneğin 17. sıradaki sembol

için “bu 17. göze git” denir; o gözün gösterdiği alana geri kalan bitler yazılır, okurken de oradan okunur. Her şey için “belki ihtiyacım olur” diye en büyük tipi kullanmak saçmadır; ileride linkli listeye geçecek şekilde kod yazmak da karmaşıktır. Hangisi uygun geliyorsa o yapılır.

16 seviyeyi geçen bir ağaç, dokümanın içinde verinin çok dengesiz dağıldığını gösterir: bazı semboller çok nadir geçmiştir ve bit değerleri ister istemez çok uzun olur. Yalnız karakterler kodlanıyorsa 16 seviyeli bir ağaç beklenmez. Ama ikili gramlar (bigram), yani var olabilecek bütün ikililer (`aa`, `ab`, …, `az`, `ba`, `bb`, …) kodlanıyorsa 26’nın ikili kombinasyonu kadar farklı sembol olur. Huffman’da her kod oluşmaz, çünkü karşımıza full binary tree çıkmaz; bu durumda ağacın derinliği ister istemez 16’yı geçer ve kodları tutan değişken belki 32 bit tanımlanmalıdır. Üçlü ya da dörtlü karakterlerde ağaç daha da derinleşir. Bu yüzden veriyi önceden tanımak önemlidir.

Hoca dinamik Huffman’ı şöyle özetledi: daha hızlıdır ama daha karmaşıktır, çünkü ağaç sürekli değişir. Bir sembolün kodu doküman içinde herhangi bir anda farklı değer alabilir: bir an 3 bitken bir an 5 bite çıkar, sonra yeniden 4 bite düşebilir. Bu dokümanın içeriğine bağlıdır.

6. Huffman Algoritmasının Basit Bir Versiyonu

Hoca, kitaplarda görülebilecek Huffman’ın farklı versiyonlarından birini anlattı (ekrandan, 00:40:00). Slayttaki maddeler:

- Basit, ancak daha az etkili bir yöntemdir.
- Eşit olasılıklı, n elemanlı değişken uzunluktaki kodlar için uygundur.
- Sembollerin olasılıkları eşit değilse etkili değildir.
- $n \times 3$ boyutunda bir tablodan oluşur; kolon değerleri sembolün kendisi, sıklığı ve kodudur.
- İkinci kolonun değeri değiştikçe satırlar yer değiştirir, ancak üçüncü kolon sabit kalır.

“Az etkili” sıkıştırma performansının daha düşük olması demektir. “Eşit olasılık” da hepsinin illa 0,2 olması değildir; birbirine yakın olasılıklar (0,1; 0,1; 0,09 gibi) varsa bu yöntem kullanılabilir.

6.1 Örnek

Alfabede dört sembol vardır: a_1, a_2, a_3, a_4. Başlangıçta count değerleri sıfırdır. Bu dört sembol için Huffman’dan çıkarılmış prefix kodlar `0`, `10`, `110`, `111`’dir.

Adım	1. satır	2. satır	3. satır	4. satır
Başlangıç	a_1 (0) `0`	a_2 (0) `10`	a_3 (0) `110`	a_4 (0) `111`
a_2 gelir	a_2 (1) `0`	a_1 (0) `10`	a_3 (0) `110`	a_4 (0) `111`
a_4 gelir	a_2 (1) `0`	a_4 (1) `10`	a_3 (0) `110`	a_1 (0) `111`
a_4 gelir	a_4 (2) `0`	a_2 (1) `10`	a_3 (0) `110`	a_1 (0) `111`

1. a_2 gelir. Kodu `10`’dur; output stream’e `10` yazılır ve a_2’nin count’u 1 olur. Olasılığı büyük olan daha az bitle ifade edilmelidir: a_2 bir kez gelmişken iki bitle, a_1 hiç gelmemişken bir bitle kodlanması adil değildir. Kodlar yerinde kalmak koşuluyla a_1 ile a_2 (frekanslarıyla birlikte) yer değiştirir.
2. a_4 gelir. Kodu `111`’dir; output stream’e üç tane 1 yazılır. a_4’ün count’u 1 olur; daha az bitle ifade edilmesi için a_1 ile yer değiştirir ve ikinci satıra çıkar.
3. Yine a_4 gelir. Kodu artık `10`’dur; `10` yazılır ve count 2 olur. En fazla a_4 kullanıldığı için 1 bitle ifade edilmelidir; a_2 ile yer değiştirip en üste çıkar.

İşlem input stream bitene kadar böyle sürer. Burada da count değerlerinin nerede tutulduğu problemi çıkar: byte’ta tutuluyorsa 255’ten sonrası nereye yazılacaktır? En üst sıradaki count değeri kontrol edilir; tutabileceği en büyük sayıya ulaştıysa bütün count değerleri yarıya bölünür, semboller yerinde kalır ve işleme devam edilir.

Hoca kullanım için şu örneği verdi: dört ya da beş rengi olan bir gray scale resim gibi az sayıda ve yakın olasılıklı sembol varsa diğer yöntemlerle uğraşmadan bu yöntem kullanılabilir.

7. Canonical Huffman Kodlama

7.1 Neden canonical Huffman?

Hoca adın neden "canonical" konduğunu bilmediğini söyledi, ama amacını anlattı. Huffman kodları oluşturulurken karşımıza bir binary tree çıkar, ama full binary tree değildir: tahtada çizdiği ağaçta aslında birçok dalda kod oluşabilirdi ama oluşmadı. Yani Huffman kodlamada oluşmayan bir sürü kod vardır.

Bu nedenle Huffman prefix kodlarıyla sıkıştırılmış dosyaların header'ları oldukça büyük olur. Önceki derste anlatılan "header'da yalnız ağacı gönderelim" fikri aslında daha sonradan geliştirilmiş bir algoritmadır. Birçok Huffman uygulamasında header'da frekans değerlerinin ve sembollerin ya da kodların ve sembollerin gönderildiği görülebilir. Semboller çok farklı bit uzunluklarında olduğu için bu header'lar çok büyük olur.

Hoca örnek verdi (ekrandan, 00:46:40): A 3 bitlik, B 5 bitlik, C 7 bitlik kodlara sahip olsun. Header'da A'nın 3 bitlik, B'nin 5 bitlik, C'nin 7 bitlik kodu arka arkaya ve arkasından A, B, C sembolleri gönderilirse karşı taraf A'nın kodunun 3 bit mi 5 bit mi olduğunu bilemez. Bu yüzden en büyük koda (7 bit) göre her koda 7 bitlik alan ayrılır. Ama A için 4 bit, B için 2 bit gereksiz olarak ayrılmıştır. Bu fazlalıklar header'ın boyunu uzatır. Header'ın boyunu küçültmek ve daha hızlı açabilmek için Huffman kodları canonical Huffman koda çevrilir.

Slayttaki maddeler:

- Huffman prefix kodları ile sıkıştırılmış dosyaların başlıklarına kıyasla, geniş alfabeli dosyalarda daha küçük dosya başlıkları oluşturmaya olanak sağlar.
- Hızlı kod dönüşümüne sahiptir.
- Huffman algoritması ile oluşturulamayıp atlanan prefix kodların oluşturulmasına izin verir.

Neden küçültüyoruz? Bazen hızlı çözüm gerekir: sıkıştırılmış bir ses, görüntü ya da arşiv dosyasına ihtiyaç olduğunda header'ın açılması beklenmek istenmez. Sese tıklandığında hemen dinlemek istenir; milisaniyeler bile önemli olabilir. Bekleme istenmiyorsa canonical Huffman kullanılır.

7.2 Oluşturma adımları

Canonical Huffman kod oluşturmanın iki aşaması vardır: önce yine Huffman kodları oluşturulur, sonra alfabedeki her karakterin Huffman kod uzunlukları kullanılarak canonical Huffman kodu oluşturulur. Slayttaki adımlar (ekrandan, 00:50:00):

1. Kodlanacak alfabede yer alan her bir karakter için Huffman prefix kod değeri geleneksel Huffman yöntemi ile oluşturulur.
2. Prefix kod uzunlukları büyükten küçüğe doğru sıralanır.
3. En uzun prefix kodun değeri L_{max} ve bu uzunlukla k_1 tane prefix kod varsa, L_{max} uzunluğunda 0'dan başlamak üzere k_1 tane ardışık binary sayı ile yeni canonical Huffman prefix kodu üretilir.
4. L_{max}'tan küçük olan bir sonraki uzunluk değeri L_{max-1} ve bu uzunlukta k_2 tane prefix kod var ise, bir önceki prefix kod değerinin ilk L_{max-1} biti alınır ve k_2 tane yeni canonical Huffman prefix kodunun her biri için 1 eklenir.
5. Bir önceki adımda yapılan işlemler büyükten küçüğe doğru tüm uzunluk değerleri için tamamlanır.

7.3 Örnek

Slayttaki soru (ekrandan, 00:56:20): Huffman algoritması kullanılarak $\{a, b, c, d, e, f, g, h\}$ alfabesi için elde edilen Huffman kodlarının uzunluk değerleri sırası ile $(5, 5, 5, 5, 3, 2, 2, 2)$ verilmiş olsun. Bu alfabe için hesaplanacak canonical Huffman kodları ne olur?

Metindeki sembollerin frekansları çıkarılmış, Huffman ağacı ve kodları elde edilmiş ve kod uzunlukları büyükten küçüğe sıralanmıştır: a, b, c, d 5 bit; e 3 bit; f, g, h 2 bit.

$L_{max} = 5$**,** $k_1 = 4$**.** 5 bit uzunluğunda 4 kod yazılacaktır ve ilk kod sıfırdan başlar: `a = 00000`. Buna 1 eklenir: `b = 00001`; 1 daha: `c = 00010`; 1 daha: `d = 00011`.

$L_{max-1} = 3$, $k_2 = 1$. En son bulunan kod `00011` alınır. Uzunluk 3 olduğu için soldaki ilk 3 bit, yani `000` alınır ve 1 eklenir: `e = 001`. Bir kod bulunacağı için bu adım biter.

$L_{max-2} = 2$, $k_3 = 3$. En son bulunan kod `001`'in soldaki ilk 2 biti `00` alınır. 1 eklenir: `f = 01`; 1 daha: `g = 10`; 1 daha: `h = 11`.

Sembol	a	b	c	d	e	f	g	h
Uzunluk	5	5	5	5	3	2	2	2
Canonical kod	`00000`	`00001`	`00010`	`00011`	`001`	`01`	`10`	`11`

> **Ders sonrası not:** Hoca 00:52:47'de dördüncü 5 bitlik kodun (`00011`) "E'nin kodu" olduğunu söyledi; slayttaki örneğe göre bu d'nin kodudur, e 3 bitlik `001` kodunu alır.

Hoca öğrencilerin sorabileceği soruyu kendisi sordu: "Hocam f, g, h zaten 2 bitti, siz de bunlara 2 bit yazdınız; e 3 bitti, 3 bit yazdınız; a-d 5 bitti, 5 bit yazdınız. Ne değişti?" Farklı olan, aynı uzunluktaki kodlar arasındaki geçişlerdir: hepsi ardışık binary sayılardır (0, 1, 2, 3 ya da 1, 2, 3). Orijinal Huffman kodlarında a, b, c'nin 5 bitlik kodları arasında böyle bir düzen yoktu ve bunlar kolayca çıkarılamazdı; canonical kodlar ise kolayca çıkarılabilir. Bulunan kodlar prefix'tir ve Kraft toplamı $4 \cdot 2^{-5} + 2^{-3} + 3 \cdot 2^{-2} = 1$'dir (python ile kontrol edildi).

> **Ek bilgi (derste anlatılmadı):** Canonical kodların ardışık olması sayesinde header'da yalnız sembollerin sırası ve her uzunlukta kaç kod olduğu (ya da her sembolün kod uzunluğu) gönderilir; alıcı kodları aynı kuralla yeniden üretir. DEFLATE (ZIP, gzip) gibi formatlar büyük olasılıkla bu yüzden Huffman tablolarını kod uzunlukları biçiminde saklar.

Hoca bu kodların günümüzde pek kullanılmadığını, çünkü sonradan ağacın kendisinin gönderilmeye başlandığını söyledi; ağaç gönderiliyorsa bu gibi şeylerle uğraşmaya gerek yoktur.

8. Modemlerde Sıkıştırma: MNP5

Aradan sonra hoca, "İstatistiksel Yöntemler 5" slaytının ikinci parçasıyla faks ve modemlerde kullanılan sıkıştırma yöntemlerine geçti. Bunlardan biri MNP (Microcom Network Protocol) olarak geçen ve modemler için geliştirilmiş yöntemlerdir; iki versiyonu vardır: MNP5 ve MNP7. En çok tercih edilen MNP5'tir, çünkü daha kolaydır. MNP7 daha hızlı ama daha karmaşıktır; bu yüzden genelde basit olan tercih edilir. Bunlar bazen modemlerin üzerinde seçimlik olarak yer alır.

8.1 İki aşama ve RLE

MNP5 iki aşamalı bir yöntemdir: önce Run-Length Coding (RLE), sonra adaptive frequency encoding. RLE'nin birkaç versiyonu daha önce görülmüştü. MNP5'te kullanılan versiyonda tekrar eden 3 ya da daha fazla byte varsa üçü arka arkaya yazılır ve dördüncü byte'a tekrar sayısı yazılır.

RLE metinlerde iyi sonuç vermezken burada neden kullanılıyor? Çünkü ne modemlerde ne de fakslarda doküman doküman olarak gider; resim olarak gider. Resim siyah-beyaz ya da gray scale olarak karşı tarafa gider ve her nokta bir piksele ya da pels'e karşılık gelir. Gönderilen şey bildiğimiz karakter değildir. Hoca örnek olarak bir I harfini çizdi: bu harf örneğin 6 scan line'dan oluşuyorsa, 6 tane siyah nokta I harfine karşılık gelir.

8.2 Token tablosu

İkinci aşama, birinci aşamanın sonuçlarını kullanır ve 6. bölümdeki basit Huffman versiyonuna benzer (ekrandan, 01:20:00). Slayttaki maddeler:

- 256 satır ve 2 sütunluk bir tablo ile başlar.
- Her bir satır 8 bitlik 0-255 arası sayıya karşılık gelir.
- Birinci kolon frekans değeridir ve başlangıç değeri 0'dır.
- İkinci kolon "token" adı verilen değişken uzunluktaki koddan oluşur.

- Her bir token 3 bitlik header (başlık) bilgisi ile başlar ve bunları kod bilgisi izler.
- Üç farklı durum dışında 2 tane 1 bitlik (0-1), 4 tane 2 bitlik (0-3), 8 tane 3 bitlik (0-7), 16 tane 4 bitlik (0-15), 32 tane 5 bitlik (0-31), 64 tane 6 bitlik (0-63) ve 128 tane 7 bitlik (0-127) koddan oluşur.
- Bahsedilen 3 farklı kodun ilk iki kodu (`000|0`, `000|1`) ve son kodu da (`111|11111110`)'dır.

Slayttaki tablodan bazı satırlar (18-25 ve 35-246 aynı kalıpla devam eder):

Byte	Token	Byte	Token	Byte	Token
0	`000|0`	8	`011|000`	32	`101|00000`
1	`000|1`	9	`011|001`	33	`101|00001`
2	`001|0`	15	`011|111`	34	`101|00010`
3	`001|1`	16	`111|0000`	247	`111|1110111`
4	`010|00`	17	`111|0001`	254	`111|1111110`
7	`010|11`	31	`111|1111`	255	`111|11111110`

Token'ın sol tarafı başlık (header), sağ tarafı kod bilgisidir. Kodlar ağaç kurularak çıkarılmaz. Header'ın gösterdiği uzunlukta sağ taraftaki kodlar 0'dan başlayıp sırayla yazılır: 1 bitlik iki kod (`0`, `1`), 2 bitlik dört kod (`00`, `01`, `10`, `11`), 3 bitlik sekiz kod (0'dan 7'ye), 4 bitlik on altı kod (0'dan 15'e), 5 bitlik 32 kod (0-31), 6 bitlik 64 kod (0-63) ve 7 bitlik 128 kod (0-127). Header kısmı ise prefix özelliğini sağlayacak biçimde yerleştirilir. 0, 1 ve 255 hiçbir kurala uymayan özel durumlardır; sıfır bitlik kod olamayacağı için 0 ve 1 için `000|0` ve `000|1` konur, en sondaki koda ise yukarıdaki kodun aynısının başına bir bit eklenir.

> **Ders sonrası not:** Hoca 01:23:56'da 16. satırda header'ın birden 7'ye (`111`) geçmesini, header'ların prefix özelliği için gray kodlamaya göre konmasıyla açıkladı. Slayttaki header dizisi (000, 001, 010, 011, ...) gray kodu değildir. Ayrıca 16-31 satırlarının header'ı `111` olursa, 128-255 satırları da `111` ile başladığı için `111|0000` (16) kodu `111|0000000`'ın öneki olur ve prefix özelliği bozulur. Büyük olasılıkla header'lar sırayla 000'dan 111'e gider ve 16-31 için doğru header `100`'dür; slayttaki `111` bir tablo hatası olmalıdır.
>
> **Ek bilgi (derste anlatılmadı):** 255'in özel kodu, 11 tane 1'den oluşan flush token'a (8.4) yer açar. Kural uygulansaydı 255'in token'ı `111|1111111` olurdu ve bu `111|11111111` flush token'ının öneki olurdu; büyük olasılıkla 255'e bu yüzden `111|11111110` verilmiştir.

8.3 Gray kodlama

Hoca header'ı açıklarken gray kodlamayı da anlattı (ekrandan, 01:25:45). Binary kodlamada (`000`, `001`, `010`, `011`, `100`, `101`, `110`, `111`) bir koddan sonrakine geçerken birden fazla bit değişebilir: `001`'den `010`'a geçerken iki bit, `011`'den `100`'e geçerken üç bit değişir. Gray kodlamada ise her geçişte yalnız bir bit değişir. `000`'dan sonra `001` gelir; sonra yalnız bir bit değişsin istenirse `011`; ardından bir bit daha değişmeli ve yukarıdakilerle aynı olmamalı, bu yüzden `010` gelir.

8.4 Adaptive frequency encoding

Slayttaki "İkinci adım" maddeleri (ekrandan, 01:30:00):

- Tablodaki frekans kolonu 0 ile başlar. Input stream'den okunan ilk byte'a karşılık gelen token tablodan çıkış stream'ine yazılır ve frekans değeri bir artırılır.
- Frekans değeri büyük olan byte, kendi üzerinde yer alan daha kısa token ile gösterilebilmesi için token'ların yeri değiştirilir (sadece token'lar).
- Frekans değeri 8 bitlik bir alanda tutulur ve her değişiklikte maksimum değere ulaşılıp ulaşılmadığı kontrol edilir. Eğer ulaşılmış ise tüm sıklık değerleri ikiye bölünür.

Örneğin 31 numaralı byte gelirse 31. satırdaki token output stream'e yazılır ve frekansı 1 artırılır. Artık frekansı 1 olduğu için daha az bitle gösterilmesi gerekir ve yer değiştirme yapılır. Bu tek seferde değil adım adım yapılır: üstündekine bakılır, ondan büyükse onunla yer değiştirilir, sonra bir üsttekiyle, ta ki doğru yere gelinceye kadar.

8.5 Örnek: `52 52 52 6`

Slayttaki örnek (ekrandan, 01:33:30): input stream'de arka arkaya 9 tane 52 vardır. RLE versiyonu "3'ten fazlaysa ilk üçünü yaz, gerisini run değeri olarak ekle" dediği için birinci aşamanın çıktısı `52 52 52 6`'dır: 3 kez 52 yazılmış, 6 da fazladan tekrar sayısıdır; $3 + 6 = 9$.

İkinci aşamada:

1. İlk 52: tablo 0'dan başladığı için 52. değer 53. satıra gelir. Slayttaki maddeye göre tabloda 53. sıradaki token stream'e yazılır. Frekansı 1 artırılır; artık en büyük frekans bu olduğu için daha az bitle ifade edilmelidir ve üsttekilerin frekansı 0 olduğu için 52 en üstteki kısa token'ı gösterir.
2. İkinci 52: yine 53. satıra bakılır; ama 52 artık yer değiştirmiş olan yeni token'ı gösterir ve o yazılır. Frekans 2 olur; kendisinden küçük frekanslı bir değer üstteyse yine yer değiştirme yapılır.
3. Üçüncü 52: aynı işlem; değişecekse kod yine değişir, değişmeyecekse aynı kod yazılır.
4. 6: bu bir run değeridir. Algoritma run değerini sembol saymaz: tablonun 7. sırasındaki token yazılır, fakat frekans değeri artırılmaz.

Slayttaki tabloya göre 52'nin başlangıç token'ı `101|10100` ($52 - 32 = 20$), 6'nın token'ı `010|10` ($6 - 4 = 2$) olur (ara adım eklendi; derste bu token değerleri okunmadı).

8.6 Çıkış ve flush token

Slayttaki maddeler (ekrandan, 01:33:30):

- Çıkış değerinde yer alan token'lar 4 ile 11 bit arasında değer alabilir. Bu çıktılar 8 bitlik paketler hâlinde çıkış stream'ine yazılır.
- En sona 11 tane birden oluşan "flush token" adı verilen kod eklenir (gerekirse 8'e tamamlamak için 1 eklenir) ve output stream sonlandırılır.
- MNP5'in iyi sonuç vermesi tamamıyla orijinal verinin içeriğine bağlıdır.

En küçük token 4 bit, en büyüğü 11 bittir. Çıkış hep 8 bitlik bloklar hâlinde yazıldığı için bazen son blok tamamlanmayabilir (örneğin en son 3 bit gelir); o zaman sonuna 1'ler eklenerek tamamlanır. Doküman bittiğinde konan 11 tane 1, dokümanın kodlanmasının bittiğini gösterir. Hoca, algoritmaların zaten verinin içeriğine göre seçildiğini hatırlattı.

> **Ders sonrası not:** Hoca 01:37:41'de token'ların "ya 4 bit ya da 11 bit" olduğunu söyledi; slayta göre token'lar 4 ile 11 bit arasında herhangi bir uzunlukta olabilir (4 ve 11 en küçük ve en büyük uzunluklardır).

8.7 Tablonun güncellenmesi: pointer'lar

Slayttaki metin (ekrandan, 01:40:00): tablonun güncellenmesi iki şekilde yapılır: (1) her sıklık değeri artırıldığında bütün tablo yeniden sıralanır; (2) tablo içerisinde pointer kullanılarak sıralama gerçekleştirilir.

Şekilde F frekans sütunu, C kod (token) sütunudur; P ve Q de pointer'lardır. Başlangıçta bütün frekanslar sıfırdır ve her satır karşısındaki kodu gösterir. a ilk geldiğinde a'nın gösterdiği kod output stream'e yazılır, a'nın frekansı 1 yapılır. Tabloda en yukarıdaki kodlar en az bitlidir ve frekansı en yüksek olanlar en az bitle kodlanmalıdır; işin esprisi budur. Bu yüzden a'nın üstündekinin frekansına bakılır: sıfırsa a'nın pointer'ı onun kodunu gösterir; bu, en üste kadar sürer. Bu sırada a'nın üzerindeki satırların pointer'ları birer aşağı iner.

Dört tane a gelirse: ilk a'da eski kod yazılır ve frekans artırılır; ikinci a'da pointer izlenerek artık üstteki kod yazılır ve frekans 2 olur; üstünde daha küçük frekanslı değer olmadığı için yerinde kalır. Ama arada başka semboller girip çıktıkça pointer'lar da değişir. Run değeri geldiğinde de ilgili satırın gösterdiği kod okunur, ama frekans değişmediği için pointer aynı yeri göstermeye devam eder. a'dan sonra gelen ikinci bir sembolün frekansı 1 olunca o da yukarı çıkar ve a'nın altındaki yere yerleşir; üçüncü bir sembol de onun altına. Sonra bu üçüncü sembol ikinci kez gelirse yukarı çıkar ve diğerleri onun altına iner.

Hoca, MNP5'in MNP7'den daha kolay olduğunu söylemesine rağmen burada bile pointer'larla, bir tür linkli listeyle işlem yapıldığına dikkat çekti.

9. MNP7: İki Boyutlu Dinamik Huffman

MNP7 daha karmaşıktır. Burada da bir Huffman kodlaması vardır, ama iki boyutlu dinamik Huffman kodlaması kullanılır; yanında yine RLE vardır. RLE bildiğimiz gibidir: 3 ve 3'ten fazla tekrar varsa yazılır; örneğin tam 3 tekrar varsa üçü yazılır ve run değeri 0 olur.

Algoritma first order Markov modeline dayanır. Hoca önceki derslerdeki modeli hatırlattı: white ve black pikseller vardı; white'tayken white ya da black, black'tayken black ya da white gelme olasılıkları vardı ve bir piksele değer verirken kendinden önceki piksele bakılıyordu. Burada da bir değerin kodu önceki değere bağlı olarak değişir. Hoca anlatımı piksel yerine karakter üzerinden yaptı: bir karakter ve kendisinden önceki karakter.

Slayttaki maddeler (ekrandan, 01:50:00):

- 256 adet, 256×2 boyutunda tablodan oluşur.
- Her satır 0 ile 255 arasındaki karakteri temsil eder.
- 256×2 boyutundaki her tablonun da ilk sütunu 0-255 arasındaki sayılardan ve ikinci sütunu da kullanım sıklıklarından oluşur ki başlangıç değeri 0'dır.
- Kodlar ayrı bir tabloda tutulur ve asla yer değiştirme yapılmaz.

MNP5'te tek bir tablo vardı: `a` geldiğinde `a`'nın bulunduğu yere gidilir, kodu alınırdı. MNP7'de ise örneğin `a` ve ardından `b` gelmişse, `b` şu an kodlanan (current) karakter, `a` bir önceki (preceding) karakterdir. Önceki karakterden ilgili tablo, şimdiki karakterden o tablodaki satır bulunur. Kesiştikleri yerdeki adres neyi gösteriyorsa ikinci tablodaki o adresin kod değeri okunur. Frekans arttıkça yer değiştirmeler de bu tablo içindeki pointer'lar üzerinden yapılır. Arka plandaki mantık MNP5 ile aynıdır, ama kodlaması çok daha zor ve karmaşıktır.

9.1 Neden iki boyut?

Hoca bir metin üzerinde saydı: metinde (kendi sayımına göre) 28 tane `a`, 5 tane `b` ve `a`'yı izleyen `b`'den (`ab`) 3 tane vardır. `a`'nın olasılığı çok daha yüksek, `b`'ninki ondan düşük, `ab`'ninki daha da düşüktür. Tek karakterler alınırsa çıkan kodlar bir türlü, ikililer alınırsa başka türlü olur; Huffman olasılık değerlerine göre oluşur. Bu dokümanda `ab` üç kez geçiyorsa `ba` üç kez geçmeyebilir. Üçlüler alınırsa olasılıklar yine farklı olur.

Varmak istenen nokta şudur: olasılık değerleri bu şekilde küçüldüğünde birbirine daha yakın kod değerleri çıkabilir ve varyans küçük olabilir. Küçük varyans, dinamik bir ortamda (ki modemler öyledir) iletim hattında kodun daha hızlı çözülmesi, gönderilenin anında açılması ve beklemelerin azalması demektir. Ama bu kodu yazmak çok meşakkatlidir. Hız çok çok önemliyse MNP7 tercih edilir; milisaniyeler o kadar önemli değilse MNP5 kullanılabilir. Modemler ve faks cihazları bu seçimi kullanıcıya bırakır.

10. Bilinen Algoritmaların Türevleri

Hoca, MNP5 ve MNP7'yi gördükten sonra genel bir gözlem paylaştı: dersin başında RLE, Huffman, aritmetik kodlama, LZW ve PPM gösterilecekti, çünkü bunlar temel ("bariz") algoritmalardır. Karşılaşılan diğer yöntemler bunların türevleri ya da karışımlarıdır. Bir modem sıkıştırmasının içinde RLE ve Huffman vardır; adı farklıdır ama özü aynıdır.

Versiyonlar dokümanın içeriğine göre değişir: bazen algoritma üzerinde oynamak, önüne ya da arkasına ekstra bir işlem eklemek gerekir ve performans artar. Algoritmalar birdenbire ortaya çıkmaz: biri MNP5'e bakar, "bunu iki boyutlu yaparsam daha hızlı çalışır" der ve yapar; ya da elindeki girdide çok fazla boşluk olduğunu görür ve bu bilgiyi algoritmaya ekleyerek performansı yükseltir. Algoritmanın adı farklı olur. Hoca, ileride görülecek LZ ailesinde de başı LZ olan ve herkesin bir şey yapıp kendi isminin baş harfini eklediği, parametrelerle oynadığı çok sayıda algoritma olduğunu, ama özlerinin aynı olduğunu söyledi.

11. Faks Sıkıştırma: Standartlar ve Çözünürlük

11.1 Standartlar

Faks üzerinden bir dokümanın gönderilmesi, o dokümanın bir resim olarak gönderilmesi demektir. Bunun için geliştirilmiş veri sıkıştırma standartları ITU (International Telecommuni-

cation Union) tarafından belirlenmiştir. T2 (Grup 1) ve T3 (Grup 2) geçmişte kullanılıyordu; bunların yerini T4 (Grup 3) ve T6 (Grup 4) aldı. T4, yani Grup 3, PSTN (Public Switched Telephone Network) ağlarında; Grup 4 ise ISDN (Integrated Services Digital Network) ağlarında kullanılır. İkisinin hızları farklıdır. Her iki yöntemde de sıkıştırma oranları 10:1 veya daha yüksektir. Hoca, sıkıştırma olmadan bir sayfanın faksla iletilmesinin çok uzun sürdüğünü, bu standartlarla sürenin bir dakikanın altına indirildiğini söyledi.

Hoca bir boyutlu ve iki boyutlu faks sıkıştırma yöntemleri olduğunu, iki boyutlunun daha karmaşık olduğunu ve onu görsellerle anlatacağını belirtti.

11.2 Pels

Faks makineleri dokümanı satır satır tarar ve satırları "pels" (picture elements) adı verilen küçük siyah beyaz noktalara çevirir (ekrandan, 02:20:00). Görüntü üzerinde çalışırken piksellerimiz vardı ve renk değerlerine karşılık geliyordu. Fakslarda da gönderilen her dosya bir resim olarak görülür, ama kavram olarak piksel yerine pels kullanılır. Pels'ler de küçük siyah-beyaz noktalardır, ama boyutları piksellerinkiyle aynı değildir. Her satıra (görüntü konusunda da geçtiği gibi) scan line denir; doküman satır satır taranır, satırlar pels'lere çevrilir ve her satır ayrı ayrı kodlanarak gönderilebilir.

11.3 Yatay ve dikey çözünürlük

Bir A4 dosyasında bir satırın kaç pels'ten, bir sütunun kaç pels'ten oluşacağını yatay ve dikey çözünürlük belirler. Slayttaki değerler:

- **Yatay çözünürlük:** her zaman 8,05 pels/mm (205 pels/inch, 1″ = 25,4 mm). 8,5″ genişliğinde taranan her satır 1.738 pels'e çevrilir. T4 standardında tarama için önerilen satır genişliği 8,2″'dir.
- **Dikey çözünürlük:** standard mode 3,85 scanline/mm; fine mode 7,7 scanline/mm; very fine mode 15,4 scanline/mm.

Hoca bir inçin 25,4 mm olduğunu hatırlattı. Bir satırın pels sayısı için ezberlemek gerekmez: satır genişliği (8,2″) inç başına pels sayısıyla (205) çarpılabilir ya da 25,4 ile çarpılıp milimetreye çevrildikten sonra milimetre başına pels sayısıyla çarpılabilir. Dikeyde görüntünün çok iyi gitmesi isteniyorsa very fine, orta kalitede fine, düşük kalitede standard mod seçilir. Standard modda bir milimetreden ortalama 3,85, yani kabaca 4 tarama satırı geçer.

> **Ders sonrası not:** Slayttaki "8,5″ genişliğinde taranan her satır 1.738 pels" ifadesinde $8{,}5 \times 205 \approx 1742$'dir. Büyük olasılıkla kastedilen, T4'te bir satır için standart olan 1728 pels'tir.

11.4 Sıkıştırmasız gönderim süresi

Slayttaki soru (ekrandan, 02:23:30): 10″ yüksekliğinde olan bir sayfada toplam pels sayısı, sıkıştırma yapılmaz ise üç farklı moddaki gönderim süresi ne olur?

Sayfanın genişliği 8,2″, yüksekliği 10″'dir. 1″ = 25,4 mm olduğu için yükseklik 254 mm'dir.

Mod	Scan lines	Pels per line	Pels per page	Süre (s)	Süre (dk)
Standard ($254 \times 3{,}85$)	978	1664	1.670M	170	2,82
Fine ($254 \times 7{,}7$)	1956	1664	3.255M	339	5,65
Very fine ($254 \times 15{,}4$)	3912	1664	6.510M	678	11,3

Satır sayıları $254 \times 3{,}85 = 977{,}9 \approx 978$, $254 \times 7{,}7 = 1955{,}8 \approx 1956$, $254 \times 15{,}4 = 3911{,}6 \approx 3912$'dir. Sayfadaki pels sayısı, satır başına pels ile satır sayısının çarpımıdır. Slayttaki sonuç cümlesi: "Sürelerdeki uzunluk, faks iletişimde veri sıkıştırmanın ne kadar önemli olduğunu göstermektedir."

Ders sonrası not: Standard mod için $978 \times 1664 = 1.627.392$'dir; slayttaki 1.670M büyük olasılıkla 1.627M olmalıdır. Fine ve very fine satırları ($1956 \times 1664 \approx 3{,}255$M, $3912 \times 1664 \approx 6{,}510$M) tutarlıdır (python ile kontrol edildi).

Ders sonrası not: Hoca 02:24:21'de "11,3 dakikada gidecekti, sıkıştırdım, 678 saniyede gönderdim" dedi. 678 saniye 11,3 dakikanın kendisidir; tablodaki iki süre sütunu aynı sürenin saniye ve dakika karşılığıdır ve slayttaki soruya göre üçü de sıkıştırma yapılmadan gönderim süreleridir.

Ek bilgi (derste anlatılmadı): Tablodaki süreler sayfa başına pels sayısının yaklaşık 9600 bit/s'ye bölünmesiyle elde edilir ($1.627.392/9600 \approx 170$ s). Tablonun alındığı kaynakta büyük olasılıkla 9600 baud'luk bir modem varsayılmıştır.

12. Grup 3 ve Modified Huffman

12.1 Kodların elde edilmesi

Bir sayfanın yüksekliği, her satırın genişliği, milimetre başına scan line ve pels sayısı bilinirse dokümandaki toplam pels sayısı bilinir. Bu pels'ler siyah-beyaz noktalardır; dolayısıyla siyah ve beyaz noktaların kullanım sayısına göre kodlar oluşturulmalıdır. Hoca Grup 3 üzerinden anlattı. Slayttaki maddeler (ekrandan, 02:28:50):

- Grup 3 kodu, birçok doküman üzerinde siyah ve beyaz pels'lerin run-length'lerinin analiz edilmesi ile elde edilir.
- Her bir run uzunluğu için Huffman algoritması ile değişken uzunlukta kodlar atanır.
- En fazla kullanılan run uzunluklarının 2, 3 ve 4 siyah pels değerleri olduğu tespit edilmiş ve bu pels değerleri için kısa kodlar atanmıştır.
- 2 ve 7 arasındaki run değerlerine sahip olan beyaz pels'lere de daha uzun kodlar atanmıştır.
- Grup 3, RLE ve Huffman kodlamanın bir bileşimidir.
- Run değerlerinin uzun olabileceği düşünüldüğünden Huffman algoritması üzerinde değişiklikler yapılmıştır. Bu sebeple Grup 3 standardı MH (Modified Huffman) olarak adlandırılır.
- 1 ile 63 run uzunluğundaki pels değerleri için termination kodlar, bu pels değerlerinin 64 katı değeri içinde make-up kodlar kullanılır.

Farklı içeriklere sahip yüzlerce doküman alınmış, içlerindeki pels'ler sayılmıştır: arka arkaya kaç siyah, kaç beyaz gelmiş (run değerleri). Olasılıkları çıkarılmış ve olasılığı yüksek olanlar daha az bitle ifade edilmiştir; bu, bilinen Huffman mantığıdır.

Ders sonrası not: Hoca 02:27:06'da 2-7 arası beyaz run'ların az görüldüğü için uzun kod aldığını söyledi. Termination tablosunda bu run'ların kodları 4 bittir (2 → `0111`, 3 → `1000`, 4 → `1011`, 5 → `1100`, 6 → `1110`, 7 → `1111`) ve tablonun en kısa beyaz kodlarıdır. Büyük olasılıkla slayttaki "daha uzun", siyah 2-4 run'larının 2-3 bitlik kodlarına (`11`, `10`, `011`) göre biraz daha uzun anlamındadır.

12.2 Termination ve make-up kodları

Modified Huffman'ın yaptığı değişiklik bulunan kodları ikiye ayırmaktır. Run değerleri 0 ile 63 arasındaysa **termination** kodları, 64 ve 64'ün katlarıysa (64, 128, 192, ...) **make-up** kodları kullanılır (ekrandan, 02:30:00). Bir satırda 1250 siyah ya da beyaz gelebilir; bütün değerler (2560'a kadar) için ayrı kod çıkarılsaydı kodlar çok uzardı. Kodları ikiye bölmek olayı optimize eder.

Termination tablosundan bazı satırlar:

Run	Beyaz	Siyah
0	`00110101`	`0000110111`
1	`000111`	`010`
2	`0111`	`11`
3	`1000`	`10`
4	`1011`	`011`

Run	Beyaz	Siyah
5	1100	0011
7	1111	00011
11	01000	0000101
12	001000	0000111
63	00110100	000001100111

Make-up tablosundan bazı satırlar: 64 → beyaz 11011, siyah 0000001111; 128 → beyaz 10010, siyah 000011001000; 2560 → 000000011111.

Run değeri 0 da olabilir. Bu, RLE'de üç tane arka arkaya geldiğinde run değerinin 0 olmasına benzer; nadir de olsa karşımıza gelir. Yazılı bir sayfa faksla gönderildiğinde yazılar siyah pels'lerle, geri kalan beyazlar beyaz pels'lerle gösterilir; sayılan şey arka arkaya kaç siyah ve kaç beyaz olduğudur. Tablodan 11 beyaz için bir kod, 11 siyah için başka bir kod, 63 beyaz ve 63 siyah için ayrı kodlar okunur.

1792'den itibaren beyaz ve siyah make-up kodları birbirinin aynıdır (slaytta "same as white from this point"). Bunun sebebi, arka arkaya bu kadar çok beyaz gelmesinin sayfanın tamamının boş olması anlamına gelmesidir; kimse faksla boş bir sayfa göndermez. Bu durumlar için ayrı kodla uğraşılmaz.

12.3 Run kodunun oluşturulması

Slayttaki kural (ekrandan, 02:37:40): bir run kodu ya tek bir termination kod veya bir veya daha fazla make-up kod ve onu izleyen termination koddan oluşur. Örneğin 65 termination kodla yazılamaz; $65 = 64 + 1$'dir. 1, beyaz ya da siyah olmasına göre termination tablosundan; 64, yine rengine göre make-up tablosundan kodlanır.

Slayttaki örnekler:

1. 12 beyaz pels: tek termination kodu, 001000 (iki tane 0 ve 1000).
2. 76 beyaz pels ($= 64 + 12$): 11011|001000.
3. 140 beyaz pels ($= 128 + 12$): 10010|001000.
4. 64 siyah pels ($= 64 + 0$): 0000001111|0000110111. Termination kısmı 0 run'ının kodudur.
5. 2561 siyah pels ($= 2560 + 1$): 000000011111|010.

> **Ders sonrası not:** Hoca 02:33:59'da 76 beyaz için önce 12'nin termination kodunun, sonra 64'ün make-up kodunun yazıldığını söyledi. Slayttaki örnek ve kurala göre önce make-up kodu (11011), sonra termination kodu (001000) gelir.

12.4 Satırların kodlanması

Slayttaki maddeler:

- Her bir tarama satırı ayrı ayrı kodlanır ve kodlar özel EOL kodu ile sonlandırılır.
- Bir satır scan edildiğinde her satırın soluna bir tane beyaz pels eklenir. Bu, alıcı tarafından kod çözüldüğünde ortadan belirsizliği kaldırmak içindir.

EOL kodu görüldüğünde birinci satır biter ve ikinci satıra geçilir. Her satır, hiç beyazla başlamasa bile, soluna bir beyaz pels konarak kodlanır. Her satırın bir beyazla başlaması ve EOL ile bitmesi kontrol mekanizması olarak kullanılır: örneğin satır sayısı kadar EOL yoksa veri bozulmuştur.

Örnek 1. 14 pels'ten oluşan satır: 3 siyah, 2 beyaz, 2 siyah, 7 beyaz. Başına bir beyaz eklenince 1w3b2w2b7wEOL olur ve kod:

000111 (1 beyaz) | 10 (3 siyah) | 0111 (2 beyaz) | 11 (2 siyah) | 1111 (7 beyaz) | EOL.

Örnek 2. 14 pels'ten oluşan satır: 2 beyaz, 5 siyah, 5 beyaz, 2 siyah. Başına bir beyaz eklenince ilk run 3 beyaz olur: 3w5b5w2bEOL, kod:

1000 (3 beyaz) | 0011 (5 siyah) | 1100 (5 beyaz) | 11 (2 siyah) | EOL.

Aradaki | ayıraçlarını hoca tablodan bakmayı kolaylaştırmak için koymuştur; orijinal çıkışta bu ayıraçlar yoktur. Bütün kodlar slayttaki termination tablosuyla karşılaştırıldı ve tuttu.

> **Ders sonrası not:** Slaytta EOL kodu `0000000001` biçiminde okunuyor. ITU T4 standardında EOL büyük olasılıkla 11 sıfır ve bir 1'den oluşan 12 bitlik `000000000001` kodudur; bu, slayttaki "kodlar 2 ile 12 bit arasındadır" ifadesiyle de uyumludur.

13. Grup 3'te Hata Bulma ve Tek Boyutlu Kodlama

Slayttaki maddeler (ekrandan, 02:40:00):

- Grup 3, kodda hata düzeltmeye sahip değildir fakat hataların çoğunu bulabilir.
- Huffman kodun doğasından dolayı iletim anında tek bir hatalı bit gönderilirse bile alıcı senkronizasyonun dışına çıkar ve hatalı bit stringi oluşur.
- Bu sebepten dolayı her satır ayrı ayrı kodlanır. Alıcı, hatayı bulmuşsa EOL koduna kadar olan bitleri atlar; böylece bir hata oluştuğunda alıcı tarafında en fazla bir satır düzeltilmeden diğer bilgiler alınır.
- Eğer alıcı satır sayısı kadar EOL kodunu göremez ise yüksek bir hata oranı olduğunu kabul eder ve işlemi iptal ederek durumu göndericiye bildirir.
- Kodlar 2 ile 12 bit arasında olduğundan dolayı alıcı 12 biti okuduktan sonra halen geçerli bir kod bulamamış ise hatayı bulur.
- Her sayfanın sonunda 6 tane EOL kodu mevcuttur.
- Bu yöntem one-dimensional coding olarak adlandırılır.
- Sıkıştırma oranı görüntüye bağlıdır. Text ve B&W görüntülerde sıkıştırma oranı yüksektir.
- Kısa run'lardan oluşan görüntülerde ise, özellikle gray scale görüntülerde başarı düşüktür.

13.1 Hata senaryoları

Hata nasıl bulunur? Örneğin 50 satır varsa 50 tane EOL olmalıdır; yoksa bir hata vardır, ama hatanın tam olarak nerede olduğu bulunamaz.

Bir satır gönderilirken EOL bozulmayabilir, ama aradaki bitlerden biri 1 yerine 0 olarak gidebilir. Bu durumda üç şey olabilir:

- Bozulan bit dizisi tabloda başka bir run'a karşılık geliyorsa alıcı hatayı hiç anlamaz ve o kodu koyar (hocanın benzetmesiyle a yerine e gelir). Yalnız o satır hatalı çizilir.
- Bozulan bit dizisi tabloda hiç yoksa alıcı oraya bir kod koyamaz; o satırda hata yapıldığını anlar, EOL'a kadar olan kısmı atlar, hatalı kabul eder ve EOL'dan itibaren işlemlerine yeniden başlar. 12 bite kadar okuyup hâlâ geçerli bir kod bulamamışsa hata vardır der. Gerekirse karşı tarafa "bir hata var, bu belgeyi bir daha gönder" der.
- EOL görülemezse alıcı, EOL'u görene kadar sonraki bitleri o satırın devamı sanar ve hata satırlara yayılır.

Bir satırdaki hata pek bir şey değiştirmez. Hoca benzetme yaptı: bir yazının ortasından bir çizgi geçmiş ya da bir scan line gitmiş olsa da metin yine okunur. EOL yakalanıyorsa bu durum kabul edilebilir. Ama birden fazla satırda bozulma varsa ya da satırlarda bir türlü kod bulunamıyorsa dokümanın yeniden gönderilmesi istenebilir. Sayfanın bittiğini anlamak için sayfanın sonuna 6 tane EOL konur; yoksa sayfanın bittiği anlaşılamaz.

13.2 Neden tek boyutlu?

Yöntem her satırı tek tek okur ve her satırı kendi içinde, diğerlerinden bağımsız olarak işler. Satır bazında işlem yapıldığı için **one-dimensional coding** (tek boyutlu kodlama) denir.

Sıkıştırma performansı yine verinin içeriğine bağlıdır. Siyah-beyaz görüntülerde başarı yüksektir. Gönderilen gray scale ya da renkli bir resimse başarı o kadar yüksek değildir, çünkü bu görüntülerde run değerleri daha küçüktür ve tablo siyah-beyaza göre çıkarılmıştır. Başarıyı artırmak için **two-dimensional** (iki boyutlu) kodlamaya geçmek gerekir. Hoca iki boyutlu kodlamanın daha karmaşık olduğunu ve bir sonraki derste bununla başlayıp ardından aritmetik kodlamaya geçeceğini söyledi.

Ek bilgi (derste anlatılmadı): İki boyutlu faks kodlamasında (T4'ün 2D modu ve T6) bir satır, bir önceki satıra göre kodlanır; ardışık satırlar çoğunlukla birbirine çok benzediği için yalnız farklar gönderilir. Grup 4'ün büyük olasılıkla daha yüksek oranlar elde etmesinin sebebi budur.

Örnek Sorular

Soru 1 (ekrandan, 00:56:20). Huffman algoritması kullanılarak $\{a, b, c, d, e, f, g, h\}$ alfabesi için elde edilen Huffman kodlarının uzunluk değerleri sırası ile $(5, 5, 5, 5, 3, 2, 2, 2)$ verilmiş olsun. Bu alfabe için hesaplanacak canonical Huffman kodları ne olur?

Çözüm (derste verildi): $L_{max} = 5$, $k_1 = 4 \rightarrow$ `a = 00000`, `b = 00001`, `c = 00010`, `d = 00011`. $L_{max-1} = 3$, $k_2 = 1 \rightarrow$ `00011`'in ilk 3 biti `000`, $+1 \rightarrow$ `e = 001`. $L_{max-2} = 2$, $k_3 = 3 \rightarrow$ `001`'in ilk 2 biti `00`, $+1 \rightarrow$ `f = 01`, `g = 10`, `h = 11`.

Soru 2 (ekrandan, 00:30:00). Dinamik Huffman algoritmasında aksayan taraflar nelerdir?

Çözüm (derste verildi): (1) Sayıcının taşması: frekans değişkeni (ör. işaretsiz 16 bit, en fazla 65535) taşabilir; kökteki frekans kontrol edilir, en büyük değere ulaşılınca yaprakların yarısı alınır (integer bölme), ara düğümler ve kök yeniden hesaplanır (930 kökli ağaç → 464). (2) Kod taşması: kodlar 16 bitte tutuluyor ve ağaç 16 seviyeden derinse alan genişletilir ya da linkli liste kullanılır.

Soru 3 (ekrandan, 00:26:30). A (1), B (2), C (2), D (2), E (9) frekanslı dinamik Huffman ağacına art arda A'lar geldiğinde ağaç nasıl değişir?

Çözüm (derste verildi): kök 16 → ikinci A ile kök 17, ağaç değişmez → üçüncü A ile (D, B) = 4, (C, A) = 5, 9 ve E = 9, kök 18 → dördüncü A ile alt ağaç 10 olur ve 9'luk E ile yer değiştirir, kök 19, bütün kodlar değişir → beşinci A ile kök 20 ve `B = 1111`.

Soru 4 (ekrandan, 00:40:00). a_1, a_2, a_3, a_4 sembolleri ve sabit `0`, `10`, `110`, `111` kodlarıyla basit Huffman versiyonunda giriş a_2, a_4, a_4 ise output stream ne olur?

Çözüm (derste verildi): $a_2 \rightarrow$ `10` (a_2 en üste çıkar); $a_4 \rightarrow$ `111` (a_4 ikinci satıra çıkar); $a_4 \rightarrow$ `10` (a_4 en üste çıkar). Çıkış: `10 111 10`.

Soru 5 (ekrandan, 01:33:30). MNP5'te dokuz tane 52'den oluşan girdi nasıl kodlanır?

Çözüm (derste verildi): RLE aşaması `52 52 52 6` verir. İkinci aşamada ilk 52 için tablonun 53. satırındaki token yazılır ve frekansı artırılır; sonraki 52'ler yer değiştirmiş olabilecek token'larla yazılır; 6 için tablonun 7. satırındaki token yazılır ama frekans artırılmaz.

Çözüm (derste verilmedi): Tablonun başlangıç durumunda 52'nin token'ı `101|10100` (32-63 aralığı, $52 - 32 = 20$), 6'nın token'ı `010|10`'dur. İlk 52'den sonra 52 en üstteki token'a (`000|0`) taşınacağı için sonraki iki 52 büyük olasılıkla `000|0` ile yazılır.

Soru 6 (ekrandan, 02:23:30). 10″ yüksekliğinde olan bir sayfada toplam pels sayısı, sıkıştırma yapılmaz ise üç farklı moddaki gönderim süresi ne olur?

Çözüm (derste verildi): yükseklik 254 mm. Standard: 978 satır, 1664 pels/satır, 170 s (2,82 dk); fine: 1956 satır, 339 s (5,65 dk); very fine: 3912 satır, 678 s (11,3 dk). Sayfa başına pels sayıları için 11.4'teki tabloya ve düzeltme notuna bakın.

Soru 7 (ekrandan, 02:37:40). 14 pels'ten oluşan `3b2w2b7w` ve `2w5b5w2b` satırlarını Grup 3 ile kodlayın.

Çözüm (derste verildi): başa bir beyaz eklenir. Birinci satır `1w3b2w2b7w` → `000111|10|0111|11|1111|EOL`. İkinci satır `3w5b5w2b` → `1000|0011|1100|11|EOL`.

Soru 8 (ekrandan, 02:37:40). 76 beyaz, 140 beyaz, 64 siyah ve 2561 siyah pels'lik run'lar nasıl kodlanır?

Çözüm (derste verildi): 76 = 64 + 12 → `11011|001000`; 140 = 128 + 12 → `10010|001000`; 64 = 64 + 0 → `0000001111|0000110111`; 2561 = 2560 + 1 → `000000011111|010`.

Dersin Toparlaması

Bu derste Huffman kodlamasının farklı kullanım biçimleri tamamlandı. Statik, yarı dinamik ve dinamik Huffman karşılaştırıldı; dinamik Huffman'da encoder ve decoder'ın ağacı escape koduyla senkron olarak nasıl kurduğu, frekanslar arttıkça ağacın nasıl yeniden düzenlendiği ve bu yapıda sayıcı ve kod taşmasının nasıl çözüldüğü gösterildi. Eşit olasılıklı semboller için basit bir tablo versiyonu ve header'ı küçültmek için kod uzunluklarından ardışık kodlar üreten canonical Huffman anlatıldı.

Dersin ikinci yarısında bu temel yöntemlerin gerçek sistemlerde nasıl birleştirildiği görüldü: modemlerde MNP5 (RLE + adaptive frequency encoding, token tablosu ve flush token) ve MNP7 (RLE + Markov modeline dayalı iki boyutlu dinamik Huffman); fakslarda ise ITU Grup 3 standardı, pels kavramı, çözünürlük modları, termination ve make-up kodlarıyla Modified Huffman ve EOL ile hata bulma. Hoca, bütün bu yöntemlerin özünde RLE ve Huffman gibi temel algoritmalar olduğunu vurguladı. Bir sonraki derste iki boyutlu faks kodlamasıyla devam edilip aritmetik kodlamaya geçilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.